

Risk Certificate

plan b3b509131878f18743922330b55916288dfd5ddd7256597b861c420b1c46d92b

BLOCKED PENDING REVIEW

A human must approve or deny.

Generated	Oct 08, 2026, 01:14:25 PM (2026-10-08T07:44:25Z)
Signature	Verified HMAC-SHA256 d1dd72b0a8104eb82b12fb04e80bd3e60c2b976ae7bf2c569adf587545b6cad8
Explanation by	Groq (sanitized findings only)

The change has been blocked pending review. The most severe issues are critical IAM policies that grant full administrative rights and a security group that leaves SSH or RDP open to the internet, which could allow unrestricted access and control. The emulator test apply succeeded.

Summary

Monthly cost change	+\$25.66 /month
Shadow run (MiniStack)	Applied: 14 resources in state, 8 found by boto3
Findings	3 critical 5 high 0 medium 0 low
Resource changes	14

Risk Flags and Fixes

CRITICAL

GO-IAM-001

aws_iam_policy.admin_star

aws_iam_policy.admin_star allows every action ("*") on every resource ("*"): full administrator access to the AWS account.

Fix: Update the policy by setting the actions field to ['s3:GetObject'] and the resource field to a specific resource ARN.

admin-star.actions = ["s3:GetObject"]

admin-star.resource = "arn:aws:s3::example-bucket/*"

CRITICAL

GO-IAMW-001

aws_iam_policy.admin_star

aws_iam_policy.admin_star widens IAM permissions to full admin (Action * on Resource *).

Fix: Modify the IAM policy to set the actions field to ['s3:GetObject'] and the resource field to a specific resource ARN.

admin-star.actions = ["s3:GetObject"]

admin-star.resource = "arn:aws:s3::example-bucket/*"

CRITICAL

GO-SG-001

aws_security_group.web_ssh

aws_security_group.web_ssh allows SSH (port 22) from 0.0.0.0/0, i.e. from anywhere on the internet.

Fix: Restrict SSH access by setting the ssh_source_cidr field to 10.0.0.0/16.

web.ssh_source_cidr = "10.0.0.0/16"

HIGH

GO-EXPOSE-001

aws_instance.web[0]

aws_instance.web[0] becomes reachable from the internet (0.0.0.0/0 or ::/0) via aws_security_group.web_ssh.

Fix: Limit instance SSH exposure by setting the ssh_source_cidr field to 10.0.0.0/16.

web.ssh_source_cidr = "10.0.0.0/16"

HIGH

GO-EXPOSE-001

aws_s3_bucket.public_bucket

aws_s3_bucket.public_bucket becomes reachable from the internet (0.0.0.0/0 or ::/0) via aws_s3_bucket_acl.public_bucket.

Fix: Disable public access by setting the public_access field to false.

public-bucket.public_access = false

HIGH

GO-EXPOSE-001

aws_security_group.web_ssh

aws_security_group.web_ssh becomes reachable from the internet (0.0.0.0/0 or ::/0) via aws_security_group.web_ssh.

Fix: Close open ports by setting the ssh_source_cidr field to 10.0.0.0/16.

web.ssh_source_cidr = "10.0.0.0/16"

HIGH

GO-RDS-001

aws_db_instance.unencrypted_db

aws_db_instance.unencrypted_db does not enable storage encryption (storage_encrypted must be true), so data at rest is unencrypted.

Fix: Enable encryption by setting the encrypted field to true.

unencrypted-db.encrypted = true

HIGH

GO-S3-001

aws_s3_bucket_acl.public_bucket

aws_s3_bucket_acl.public_bucket sets the canned ACL "public-read": anyone on the internet can read every object in this bucket.

Fix: Make the bucket private by setting the public_access field to false.

public-bucket.public_access = false

Cost Breakdown (monthly)

Resource	USD/month	Usage assumptions
aws_db_instance.unencrypted_db	15.44	none
aws_ebs_volume.web[0]	1.60	none
aws_instance.web[0]	8.39	none
aws_s3_bucket.public_bucket	0.23	storage_gb=10, monthly_put_requests=0, monthly_get_requests=0

Shadow Environment: Resources Found in MiniStack

aws_db_instance	ghostops-unencrypted-db (db-179746E48E0A4464A601)
aws_ebs_volume	vol-006a9a8aba8ff6c46
aws_ebs_volume	vol-311b810a464351ebf
aws_iam_policy	arn:aws:iam::000000000000:policy/ghostops-admin-star-policy
aws_iam_role	ghostops-admin-star-role
aws_instance	i-79bd7bf05f27ef0a6
aws_s3_bucket	ghostops-public-bucket-650b8b
aws_security_group	sg-bd4148333708c3c6f

Architecture

```
web ec2 · shadow + static
instance_type="t3.micro" count=1 volume_gb=20 ssh_source_cidr="0.0.0.0/0"

admin-star iam · shadow + static
trusted_service="ec2.amazonaws.com" actions=["*"] resource="*"

public-bucket s3 · shadow + static
public_access=true versioning=true encryption=true size_gb=10

unencrypted-db rds · shadow + static
engine="postgres" instance_class="db.t3.micro" storage_gb=20 multi_az=false encrypted=false publicly_accessible=false
```

Resource Changes

create aws_db_instance.unencrypted_db	create aws_s3_bucket_ownership_controls.public_bucket
create aws_ebs_volume.web[0]	create aws_s3_bucket_public_access_block.public_bucket
create aws_iam_policy.admin_star	create
create aws_iam_role.admin_star	aws_s3_bucket_server_side_encryption_configuration.public_bucket
create aws_iam_role_policy_attachment.admin_star	create aws_s3_bucket_versioning.public_bucket
create aws_instance.web[0]	create aws_security_group.web_ssh
create aws_s3_bucket.public_bucket	create aws_volume_attachment.web[0]
create aws_s3_bucket_acl.public_bucket	

Review

Awaiting review: no decision recorded.

Appendix: Generated Terraform

```
# Generated by GhostOps from the service catalog: ec2:web, iam:admin-star, s3:public-bucket, rds:unencrypted-db
# Targets MiniStack (local AWS emulator) with fake credentials. For real AWS,
# replace the provider block and the placeholder AMI.
```

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

variable "ministack_endpoint" {
  type    = string
  default = "http://localhost:4566"
}

provider "aws" {
  region                  = "us-east-1"
  access_key              = "test"
  secret_key              = "test"
  s3_use_path_style       = true
  skip_credentials_validation = true
  skip_metadata_api_check  = true
  skip_requesting_account_id = true

  endpoints {
    cloudwatch = var.ministack_endpoint
    dynamodb   = var.ministack_endpoint
    ec2         = var.ministack_endpoint
    elbv2       = var.ministack_endpoint
    iam         = var.ministack_endpoint
    lambda      = var.ministack_endpoint
    rds         = var.ministack_endpoint
    s3          = var.ministack_endpoint
    sts         = var.ministack_endpoint
  }
}

# --- ec2: web -----

resource "aws_security_group" "web_ssh" {
  name        = "ghostops-web-ssh"
  description = "SSH access for web"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name           = "ghostops-web-ssh"
    ManagedBy      = "ghostops"
    GhostOpsService = "ec2"
  }
}

resource "aws_instance" "web" {
  count          = 1
  ami           = "ami-12345678" # placeholder: replace for real AWS
  instance_type = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web_ssh.id]

  tags = {
    Name           = "ghostops-web-${count.index}"
    ManagedBy      = "ghostops"
    GhostOpsService = "ec2"
  }
}

# Data volume per instance. A separate EBS volume rather than root_block_device:
# MiniStack instances have no root EBS volume, so the AWS provider fails to read
# root_block_device back ("collecting instance settings: empty result").
resource "aws_ebs_volume" "web" {
  count          = 1
  availability_zone = aws_instance.web[count.index].availability_zone
  size           = 20
  type           = "gp3"
  encrypted      = true
}
```

```

tags = {
    Name           = "ghostops-web-data-${count.index}"
    ManagedBy      = "ghostops"
    GhostOpsService = "ec2"
}
}

resource "aws_volume_attachment" "web" {
    count          = 1
    device_name    = "/dev/sdf"
    volume_id      = aws_ebs_volume.web[count.index].id
    instance_id    = aws_instance.web[count.index].id
}

# --- iam: admin-star -----

resource "aws_iam_role" "admin_star" {
    name = "ghostops-admin-star-role"
    assume_role_policy = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Effect = "Allow"
            Principal = { Service = "ec2.amazonaws.com" }
            Action   = "sts:AssumeRole"
        }]
    })
    tags = {
        Name           = "ghostops-admin-star-role"
        ManagedBy      = "ghostops"
        GhostOpsService = "iam"
    }
}

resource "aws_iam_policy" "admin_star" {
    name = "ghostops-admin-star-policy"
    policy = jsonencode({
        Version = "2012-10-17"
        Statement = [{
            Effect = "Allow"
            Action = ["*"]
            Resource = "*"
        }]
    })
    tags = {
        Name           = "ghostops-admin-star-policy"
        ManagedBy      = "ghostops"
        GhostOpsService = "iam"
    }
}

resource "aws_iam_role_policy_attachment" "admin_star" {
    role       = aws_iam_role.admin_star.name
    policy_arn = aws_iam_policy.admin_star.arn
}

# --- s3: public-bucket -----

resource "aws_s3_bucket" "public_bucket" {
    bucket = "ghostops-public-bucket-650b8b"
    tags = {
        Name           = "ghostops-public-bucket-650b8b"
        ManagedBy      = "ghostops"
        GhostOpsService = "s3"
        ExpectedSizeGB  = "10"
    }
}

resource "aws_s3_bucket_versioning" "public_bucket" {
    bucket = aws_s3_bucket.public_bucket.id
    versioning_configuration {
        status = "Enabled"
    }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "public_bucket" {
    bucket = aws_s3_bucket.public_bucket.id
    rule {
        apply_server_side_encryption_by_default {
            sse_algorithm = "AES256"
        }
    }
}

resource "aws_s3_bucket_public_access_block" "public_bucket" {
    bucket = aws_s3_bucket.public_bucket.id
    block_public_acls       = false
    block_public_policy     = false
    ignore_public_acls     = false
    restrict_public_buckets = false
}

resource "aws_s3_bucket_ownership_controls" "public_bucket" {
    bucket = aws_s3_bucket.public_bucket.id
    rule {
        object_ownership = "BucketOwnerPreferred"
    }
}

```

```

}

resource "aws_s3_bucket_acl" "public_bucket" {
  bucket      = aws_s3_bucket.public_bucket.id
  acl         = "public-read"
  depends_on = [aws_s3_bucket_ownership_controls.public_bucket, aws_s3_bucket_public_access_block.public_bucket]
}

# --- rds: unencrypted-db -----
resource "aws_db_instance" "unencrypted_db" {
  identifier      = "ghostops-unencrypted-db"
  engine          = "postgres"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  multi_az        = false
  storage_encrypted = false
  publicly_accessible = false
  username        = "ghostops"
  manage_master_user_password = true # password generated and kept in Secrets Manager
  skip_final_snapshot = true
  tags = {
    Name           = "ghostops-unencrypted-db"
    ManagedBy      = "ghostops"
    GhostOpsService = "rds"
  }
}

```

Generated by GhostOps. Verify this certificate with GET /verify/b3b509131878f18743922330b55916288dfd5ddd7256597b861c420b1c46d92b. GhostOps never applies changes to a real cloud account.